

Roteiro do Projeto: Sistema Acadêmico Distribuído

Disciplina de Sistemas Distribuídos. UFOPA. Projeto final (3ª avaliação).

Este documento é a referência única do projeto: contém a arquitetura, a especificação completa dos três serviços, o contrato de comunicação entre eles, o mapa de decisões e casos extremos, o plano de implementação por fases (com prompts prontos para o Antigravity), a estratégia de testes, a estrutura do README e o roteiro de apresentação. A ideia é que vocês não precisem decidir mais nada durante a implementação, apenas executar o que já está fechado aqui.

1. Princípios da arquitetura

Quatro ideias governam todas as decisões deste documento, e vale ter clareza sobre elas porque é exatamente isso que o professor vai sondar na apresentação.

Banco de dados por serviço. Cada microsserviço é o único dono dos seus dados. Nenhum serviço acessa o banco SQLite de outro diretamente; toda troca de informação passa pela API REST do dono do dado. Isso é o que torna os serviços de fato independentes, e não apenas três pastas de código separadas.

Comunicação síncrona e desacoplada por contrato. Os serviços conversam por HTTP usando JSON, e o que importa é o contrato (rotas, formato de entrada e saída, códigos de status), não a implementação interna de quem responde. Por isso a especificação da seção 3 é tratada como definitiva antes de qualquer linha de código: ela é o que permite implementar Alunos sem saber nada sobre como Matrículas será construído depois.

Falha de um serviço não pode derrubar os outros. Se o serviço de Alunos cair, o serviço de Disciplinas continua respondendo normalmente, e o serviço de Matrículas precisa degradar de forma controlada (responder 503 com mensagem clara) em vez de travar ou estourar uma exceção sem tratamento. Esse comportamento é o que diferencia um sistema distribuído de três APIs que só por acaso rodam juntas.

Independência de execução e configuração. Cada serviço sobe com seu próprio comando uvicorn, sua própria porta e seu próprio arquivo de banco. Endereços de outros serviços vêm de variável de ambiente, nunca hardcoded, para que a configuração mude (de localhost para nome de container, por exemplo) sem tocar no código.

2. Estrutura do repositório

Monorepo com uma pasta por serviço. Para um projeto de disciplina com três serviços pequenos e uma única apresentação, um repositório multi-repo só adicionaria complexidade de coordenação sem benefício real.

sistema-academico-distribuido/

```
|— alunos-service/
|   |— main.py
|   |— models.py
|   |— schemas.py
|   |— database.py
|   |— requirements.txt
|   |— Dockerfile
|   └— .env.example
|— disciplinas-service/
|   |— main.py
|   |— models.py
|   |— schemas.py
|   |— database.py
|   |— requirements.txt
|   |— Dockerfile
|   └— .env.example
|— matriculas-service/
```

```
| |— main.py
| |— models.py
| |— schemas.py
| |— database.py
| |— clients.py      # chamadas HTTP para os outros serviços
| |— requirements.txt
| |— Dockerfile
| |— .env.example
|— docker-compose.yml
|— README.md
|— tests/
    |— test_integracao.py # opcional, ver seção 7
```

Os três serviços compartilham o mesmo esqueleto interno (`main.py`, `models.py`, `schemas.py`, `database.py`), o que reduz a curva de aprendizado: quem entender Alunos entende Disciplinas sem esforço adicional.

3. Especificação dos serviços

ORM assumido: SQLAlchemy, por ser a combinação padrão com FastAPI e SQLite. Se preferirem `sqlite3` puro, a troca fica isolada em `database.py` e não afeta nenhum contrato abaixo.

3.1 Serviço de Alunos (porta 8001, `alunos.db`)

Modelo

Campo	Tipo	Regra
id	inteiro	gerado pelo banco

Campo	Tipo	Regra
nome	string	obrigatório, 3 a 100 caracteres
email	string	obrigatório, formato de e-mail válido, único
curso	string	opcional
ativo	booleano	default true

Endpoints

Método	Rota	Status de sucesso	Erros possíveis
POST	/alunos	201	409 se e-mail já existe; 422 se validação falhar
GET	/alunos	200	(suporta ?skip=&limit= para paginação opcional)
GET	/alunos/{id}	200	404 se não existir
PUT	/alunos/{id}	200	404 se não existir; 409 se novo e-mail colidir com outro aluno
DELETE	/alunos/{id}	200	404 se não existir

Regra de negócio importante: DELETE é soft delete, marca **ativo=false** em vez de remover a linha. Um hard delete deixaria órfã qualquer matrícula que reference esse aluno, já que Matrículas guarda só o id.

3.2 Serviço de Disciplinas (porta 8002, **disciplinas.db**)

Modelo

Campo	Tipo	Regra
id	inteiro	gerado pelo banco
nome	string	obrigatório, 3 a 100 caracteres
codigo	string	obrigatório, único, ex: SD001
carga_horaria	inteiro	obrigatório, maior que zero
vagas	inteiro	obrigatório, maior ou igual a zero
ativo	booleano	default true

Endpoints: mesmo padrão de Alunos (POST, GET lista, GET por id, PUT, DELETE), trocando **email** único por **codigo** único, e soft delete pelo mesmo motivo.

3.3 Serviço de Matrículas (porta 8003, **matriculas.db**)

Modelo

Campo	Tipo	Regra
id	inteiro	gerado pelo banco
aluno_id	inteiro	obrigatório
disciplina_id	inteiro	obrigatório
data_matricula	datetime	preenchida automaticamente na criação
status	string	ativa ou cancelada , default ativa

Restrição de banco: índice único composto em (**aluno_id**, **disciplina_id**, **status**) para a condição de **status = 'ativa'**, o que impede duplicidade mesmo sob duas requisições concorrentes, sem depender só da validação em código.

Endpoints

Método	Rota	Status de sucesso	Erros possíveis
POST	/matriculas	201	404 se aluno ou disciplina não existir; 400 se algum estiver inativo; 409 se matrícula ativa já existe; 503 se outro serviço estiver fora do ar
GET	/matriculas	200	(paginação opcional)
GET	/matriculas/{id}	200	404 se não existir
GET	/matriculas/{id}/completa	200	agrega dados do aluno e da disciplina via chamada HTTP; é a rota que melhor demonstra comunicação entre serviços
PUT	/matriculas/{id}	200	404 se não existir; 400 se já estiver cancelada e tentar cancelar de novo
DELETE	/matriculas/{id}	200	404 se não existir

4. Contrato de comunicação entre serviços

Ao receber `POST /matriculas`, a sequência é sempre a mesma: primeiro `GET {ALUNOS_SERVICE_URL}/alunos/{aluno_id}`, depois `GET {DISCIPLINAS_SERVICE_URL}/disciplinas/{disciplina_id}`. Só depois de confirmar que os dois existem e estão com `ativo=true` é que o registro é gravado no banco local de Matrículas.

Tratamento de erro por cenário:

- **404 de um dos dois serviços:** a matrícula é rejeitada com 404 e mensagem indicando qual recurso (aluno ou disciplina) não foi encontrado.
- **Recurso encontrado mas `ativo=false`:** rejeitada com 400, já que existir e estar disponível para matrícula são coisas diferentes.
- **Timeout ou erro de conexão:** o cliente HTTP (httpx) precisa de um timeout explícito, por exemplo 5 segundos; ao capturar `TimeoutException` ou `ConnectError`, a resposta é 503 indicando qual serviço está indisponível, nunca uma exceção não tratada subindo até o cliente.
- **As duas chamadas falham por motivos diferentes:** reportar o primeiro erro encontrado é suficiente; não é necessário agregar múltiplos erros para o escopo da disciplina.

URLs de outros serviços vêm de variável de ambiente (`ALUNOS_SERVICE_URL`, `DISCIPLINAS_SERVICE_URL`), com `http://localhost:8001` e `http://localhost:8002` como padrão em desenvolvimento. Isso evita reescrever código quando migrarem para Docker Compose, onde o valor passa a ser o nome do serviço no compose em vez de `localhost`.

Decisão deliberada de escopo: Matrículas só faz leitura (GET) nos outros dois serviços, nunca escrita. Decrementar `vagas` em Disciplinas no momento da matrícula pareceria natural, mas introduz uma escrita cross-service que pode falhar depois que a matrícula local já foi criada, abrindo um problema de consistência distribuída (transação distribuída, compensação, etc.) que está fora do escopo de uma disciplina cujo foco declarado é comunicação entre serviços, não consistência transacional. A recomendação é deixar isso fora do código e citar como melhoria futura no README, o que também sinaliza ao professor que a omissão foi deliberada, não desconhecida.

5. Mapa de decisões e casos extremos

Tabela pensada para responder de cabeça qualquer pergunta do tipo "por que vocês fizeram assim" na apresentação.

Situação	Decisão	Por quê
Excluir aluno ou disciplina referenciado em matrícula	Soft delete (<code>ativo=false</code>)	Hard delete deixaria matrículas órfãs, já que Matrículas só guarda o id

Situação	Decisão	Por quê
Duas requisições simultâneas matriculando o mesmo aluno na mesma disciplina	Índice único composto no banco	Validação em código sozinha tem race condition; a garantia real vem do banco
Aluno ou disciplina inativos no momento da matrícula	Rejeitar com 400, não 409	Existir e estar disponível são condições diferentes; o código de status deve refletir isso
Serviço dependente fora do ar	Capturar erro de timeout/conexão, responder 503	Uma exceção não tratada subindo ao cliente não é aceitável em um sistema que se propõe distribuído
Controle de vagas disponíveis	Fora do escopo da implementação, citado como trabalho futuro	Evita escrita cross-service e o problema de consistência distribuída que ela cria
Cancelar matrícula já cancelada	400, operação não é idempotente	Demonstra validação de regra de negócio, não só de schema
Autenticação e autorização	Fora do escopo, citado no README	Não consta nos requisitos obrigatórios; mencionar evita parecer omissão não percebida
Paginação em listagens	Opcional via <code>skip/limit</code>	Não obrigatório, mas barato de implementar e mostra cuidado de engenharia

6. Plano de execução por fases

Cada fase tem um objetivo, um critério de pronto e um prompt sugerido para o Antigravity, já redigido como o agente espera receber: objetivo em linguagem natural, com as regras de negócio explícitas. Cole o prompt, deixe o agente gerar o Implementation Plan, revise antes de aprovar, e só então deixe executar.

Fase 0: Setup do ambiente

Objetivo: ambiente Python pronto para os três serviços. Critério de pronto: `fastapi`, `uvicorn`, `sqlalchemy`, `httpx`, `pydantic[email]` instalados e um "hello world" em FastAPI rodando na porta 8000.

Fase 1: Serviço de Alunos

Critério de pronto: os cinco endpoints da seção 3.1 funcionando, testáveis via `/docs`, com soft delete e validação de e-mail único.

Prompt sugerido:

Crie um microserviço FastAPI na pasta `alunos-service/`, com SQLite próprio (`alunos.db`) via SQLAlchemy. Modelo Aluno com campos: id (autoincremento), nome (string, 3 a 100 caracteres, obrigatório), email (formato de e-mail válido, único, obrigatório), curso (string, opcional), ativo (booleano, default true). Exponha as rotas REST: POST `/alunos` (retorna 409 se e-mail duplicado), GET `/alunos` (com paginação opcional via skip e limit), GET `/alunos/{id}` (404 se não existir), PUT `/alunos/{id}` (atualiza todos os campos, 404 se não existir, 409 se novo e-mail colidir), DELETE `/alunos/{id}` (soft delete: marca ativo=false, não remove do banco, 404 se não existir). Adicione GET `/health` retornando `{"status": "ok", "service": "alunos"}`. Use Pydantic para validação de entrada e saída e HTTPException com mensagens claras para erros. Configure logging básico das requisições. Rode com uvicorn na porta 8001 e valide os cinco endpoints em `/docs`.

Fase 2: Serviço de Disciplinas

Critério de pronto: mesmo padrão da Fase 1, com `codigo` único no lugar de `email`.

Prompt sugerido:

Replique a estrutura do `alunos-service` para criar `disciplinas-service` na pasta `disciplinas-service/`, SQLite próprio (`disciplinas.db`). Modelo Disciplina: id, nome (obrigatório, 3 a 100 caracteres), codigo (obrigatório, único, até 10 caracteres), carga_horaria (inteiro maior que zero), vagas (inteiro maior ou igual a zero), ativo (booleano, default true). Mesmas cinco rotas REST do padrão de Alunos, com 409 em código duplicado em vez de e-mail duplicado, e soft delete em DELETE. GET `/health` retornando `{"status": "ok", "service": "disciplinas"}`. Rode na porta 8002.

Fase 3: Serviço de Matrículas, em três etapas

Não peça tudo de uma vez nessa fase: a comunicação entre serviços é o ponto mais sensível do projeto, e fica mais fácil de depurar dividindo em três prompts sequenciais.

3a. Modelo e schema, sem comunicação ainda:

Crie matriculas-service na pasta `matriculas-service/`, SQLite próprio (`matriculas.db`). Modelo Matricula: id, aluno_id (inteiro), disciplina_id (inteiro), data_matricula (datetime, preenchida automaticamente na criação), status (string, "ativa" ou "cancelada", default "ativa"). Adicione um índice único composto em (aluno_id, disciplina_id) para registros com status "ativa". Implemente por enquanto GET /matriculas, GET /matriculas/{id} e PUT /matriculas/{id} (atualiza status, retorna 400 se já estiver cancelada). Ainda não implemente POST, isso vem na próxima etapa.

3b. Integração HTTP real:

Implemente POST /matriculas no matriculas-service. Antes de criar o registro, faça GET em {ALUNOS_SERVICE_URL}/alunos/{aluno_id} e GET em {DISCIPLINAS_SERVICE_URL}/disciplinas/{disciplina_id}, lendo essas URLs de variáveis de ambiente com <http://localhost:8001> e <http://localhost:8002> como padrão. Use httpx com timeout de 5 segundos. Se algum dos dois retornar 404, responda 404 indicando qual recurso não foi encontrado. Se algum existir mas tiver ativo=false, responda 400. Se a chamada HTTP der timeout ou erro de conexão, responda 503 indicando qual serviço está indisponível. Se já existir matrícula ativa para esse aluno e disciplina, responda 409. Coloque a lógica de chamada aos outros serviços em um arquivo clients.py separado do main.py.

3c. Rota de agregação e observabilidade:

Implemente GET /matriculas/{id}/completa no matriculas-service, retornando o registro de matrícula com os dados completos do aluno e da disciplina obtidos via chamada HTTP aos respectivos serviços no momento da requisição. Se um dos serviços estiver indisponível nesse momento, retorne 503. Adicione GET /health retornando {"status": "ok", "service": "matriculas"}. Adicione logging de cada chamada HTTP feita a outro serviço, incluindo tempo de resposta, para facilitar demonstração e depuração. Rode na porta 8003.

Fase 4: Teste de integração ponta a ponta

Critério de pronto: os três serviços rodando simultaneamente, e um roteiro manual (ou script) exercitando: criar aluno, criar disciplina, matricular com sucesso, tentar matricular aluno

inexistente (espera 404), tentar matricular duplicado (espera 409), desligar o serviço de Alunos e tentar matricular (espera 503), religar e confirmar que volta a funcionar. Ver seção 7 para o roteiro completo.

Fase 5: Docker e Docker Compose (opcional, recomendado)

Critério de pronto: `docker-compose up` sobe os três serviços, cada um com seu volume de SQLite persistido, e Matrículas consegue chamar Alunos e Disciplinas pelo nome do serviço no compose em vez de `localhost`. Embora opcional na lista de tecnologias, essa fase reforça diretamente o critério "Arquitetura Distribuída" (20% da nota), porque demonstra de forma visual que cada serviço é uma unidade de execução independente.

Fase 6: README

Ver seção 8 para a estrutura completa.

Fase 7: Roteiro de apresentação e ensaio

Ver seção 9.

7. Estratégia de testes

Manual, via `/docs` ou `curl`, é suficiente para o escopo da disciplina e é o que vocês vão demonstrar ao vivo. Roteiro mínimo, na ordem:

1. Subir os três serviços, confirmar `/health` de cada um.
2. POST em `/alunos` e em `/disciplinas`, confirmar 201.
3. POST em `/matriculas` com os ids criados, confirmar 201.
4. GET em `/matriculas/{id}/completa`, confirmar que retorna dados agregados dos três serviços.
5. POST em `/matriculas` repetindo o mesmo par aluno/disciplina, confirmar 409.
6. POST em `/matriculas` com `aluno_id` inexistente, confirmar 404.
7. Derrubar o serviço de Alunos (Ctrl+C no terminal dele), repetir o POST de matrícula, confirmar 503.
8. Religar o serviço de Alunos, repetir o passo 3 com um par novo, confirmar que volta a funcionar.

Esse roteiro, executado ao vivo na apresentação, é a evidência mais direta de "comunicação entre serviços" que existe, vale literalmente 20% da nota sozinho.

Testes automatizados (opcional, mas conta como diferencial nas tecnologias opcionais):
`pytest` com `TestClient` do FastAPI para testes unitários de Alunos e Disciplinas

isoladamente; para Matrículas, um teste de integração que sobe os três serviços via subprocess ou usa `respX/mock` para simular as respostas de Alunos e Disciplinas sem precisar deles rodando.

8. Estrutura do README

Conteúdo mínimo, na ordem que melhor responde aos critérios de "Documentação" (10%) e "Arquitetura Distribuída" (20%):

1. Nome do projeto e disciplina.
 2. Diagrama de arquitetura (pode ser o diagrama ASCII da seção 1, ou exportado do Mermaid).
 3. Tecnologias usadas.
 4. Como executar cada serviço (comandos exatos, incluindo variáveis de ambiente necessárias).
 5. Como executar via Docker Compose, se implementado.
 6. Exemplos de chamadas HTTP (request e response) dos fluxos principais: criar aluno, criar matrícula, erro de matrícula duplicada.
 7. Decisões de design relevantes: por que banco por serviço, por que não decrementar vagas automaticamente, por que soft delete.
 8. Limitações conhecidas e trabalho futuro: autenticação, controle automático de vagas, versionamento de API.
-

9. Roteiro de apresentação

Distribuição sugerida (ajustem ao tamanho real do grupo, mas mantendo todos falando, já que é requisito do projeto): uma pessoa apresenta a arquitetura geral e o diagrama, uma pessoa demonstra Alunos e Disciplinas isoladamente, uma pessoa demonstra Matrículas e a comunicação entre serviços (incluindo o caso de falha), e a última parte (perguntas do professor) é respondida por quem estiver mais confortável com cada tópico.

Sequência de demonstração:

1. Mostrar os três serviços subindo em terminais separados, cada um respondendo em `/health`: isso já demonstra "cada microserviço executa independentemente" de forma visual e imediata.
2. CRUD básico de Alunos e Disciplinas via `/docs`.
3. Criação de matrícula com sucesso, mostrando no terminal os logs das duas chamadas HTTP saindo do serviço de Matrículas.

4. Caso de erro: tentar matricular aluno inexistente, mostrando o 404.
5. Caso de resiliência: derrubar o serviço de Alunos ao vivo, tentar matricular, mostrar o 503, religar o serviço e repetir com sucesso. Esse passo isolado costuma impressionar mais que qualquer outro, porque mostra entendimento de tolerância a falhas, não só CRUD funcionando.

Perguntas prováveis do professor e onde está a resposta neste documento: "por que banco separado por serviço" (seção 1), "o que acontece se um serviço cair" (seções 4 e 9, passo 5), "por que não descontar vagas automaticamente" (seção 4, decisão deliberada), "como vocês evitam matrícula duplicada" (seção 3.3, índice único composto).

10. Checklist final cruzado com os critérios de avaliação

Critério	Peso	Onde este documento garante isso
Funcionamento	20%	Seções 3 e 7: especificação completa e roteiro de teste ponta a ponta
Arquitetura Distribuída	20%	Seções 1, 2 e 5: banco por serviço, independência de execução, decisões justificadas
Comunicação entre serviços	20%	Seção 4 e roteiro de apresentação passo 3 e 5: contrato de chamadas e demonstração de falha controlada
Qualidade do código	15%	Prompts da seção 6 já embutem validação, tratamento de erro e separação de responsabilidades (clients.py)
Documentação	10%	Seção 8: estrutura de README cobrindo arquitetura, execução e decisões

Critério	Peso	Onde este documento garante isso
Apresentação	15%	Seção 9: roteiro com distribuição de falas e respostas preparadas para perguntas previsíveis

Com este documento fechado, a Fase 1 (serviço de Alunos) está pronta para entrar no Antigravity agora.